

NYC Taxi AI Analytics Project

One-File Submission Guide: Code, Requirements, Directions, Model, and Outputs

Team members: Soumith Kumar Ananthula, Varun Reddy Beesam, Vivek Doppalapudi

This PDF consolidates the repository instructions, model explanation, dependency requirements, preprocessing summary, evaluation metrics, prediction usage, and core source code for submission review.

1. Project Overview

The project builds a working AI-driven analytics solution using Python and the NYC Taxi and Limousine Commission yellow taxi trip records. The model predicts whether a new taxi trip is likely to last 30 minutes or longer.

- Problem type: supervised binary classification.
- Prediction target: long_trip, where 1 means 30+ minutes and 0 means under 30 minutes.
- Model: optimized histogram gradient boosting with a tuned precision-focused threshold.
- Use case: operational planning for long-trip risk, not ride denial or punitive decision-making.
- Repository: https://github.com/soumithkumara/project_day_1

2. Repository Structure

```
project_day_1/  
|-- data/README.md  
|-- notebooks/  
|   |-- nyc_taxi_long_trip_model.ipynb  
|   |-- predict_new_taxi_trip.ipynb  
|-- reports/  
|   |-- NYC_Taxi_AI_Analytics_APA7_Report.docx  
|   |-- assignment_report.md  
|   |-- metrics.json  
|   |-- preprocessing_audit.csv  
|   |-- feature_engineering_summary.csv  
|   |-- target_distribution.csv  
|   |-- model_metrics.csv  
|   |-- model_coefficients.csv  
|   |-- model_feature_importance.csv  
|   |-- tuning_results.csv  
|   |-- figures/  
|-- src/nyc_taxi_long_trip_model.py  
|-- scripts/  
|   |-- create_apa7_report_docx.py  
|   |-- create_submission_pdf.py  
|-- requirements.txt  
|-- README.md
```

3. Software Requirements

Install the Python dependencies below when running locally. In Colab, the notebooks verify packages and avoid unnecessary NumPy upgrades.

```
pandas==3.0.3  
numpy==2.4.6  
matplotlib==3.11.0  
seaborn==0.13.2  
scikit-learn==1.9.0  
pyarrow==24.0.0  
joblib==1.5.3  
reportlab==5.0.0
```

4. How to Run the Main Analysis in Google Colab

Use these commands in a Colab notebook if you are running from a blank session:

```
!git clone --branch project_day_2 https://github.com/soumithkumara/project_day_1.git
%cd project_day_1
!pip install -r requirements.txt
!python src/nyc_taxi_long_trip_model.py
```

Recommended Colab notebooks:

- notebooks/nyc_taxi_long_trip_model.ipynb - trains the model and displays preprocessing, metrics, and charts.
- notebooks/predict_new_taxi_trip.ipynb - loads/trains the model and predicts a new taxi trip.

5. Dataset and Preprocessing Summary

The official January 2024 yellow taxi parquet file contained 2,964,624 raw rows. After filtering invalid or unrealistic records, 2,684,599 rows remained. A deterministic 120,000-row sample was used for modeling so the project runs quickly and reproducibly.

Step	Rows removed	Rows after
Kept trips with pickup timestamps in January 2024.	18	2,964,606
Removed trips shorter than 1 minute or longer than 180 minutes.	37,104	2,927,502
Removed trips with zero, near-zero, negative, or extreme distances.	38,743	2,888,759
Kept trips with passenger_count from 1 to 6.	147,459	2,741,300
Removed trips with non-positive fare_amount.	29,647	2,711,653
Kept standard TLC rate codes 1 through 6.	27,054	2,684,599
Selected deterministic modeling sample with random_state=42.	2,564,599	120,000

6. Feature Engineering

Feature	Type	Purpose
trip_duration_min	derived numeric	Used for cleaning and to create the long_trip target; not used as a model input.
pickup_hour	derived numeric	Captures daily travel and congestion cycles.
pickup_dayofweek	derived numeric	Captures weekday/weekend and commute-pattern differences.
is_weekend	derived binary	Flags weekend operating patterns.
airport_trip	derived binary	Flags trips involving major airport zones.
long_trip	target binary	Prediction target: 1 for long trips, 0 otherwise.
VendorID, RatecodeID, PULocationID, DOLocationID	Do not treat as ordered	Prevents ID values from being treated as ordered numeric quantities.

7. Model Details and Metrics

The final AI model is an optimized histogram gradient boosting classifier. Numeric features are median-imputed, categorical features are ordinal-encoded, and a validation-selected probability threshold is used to predict class 0 for under 30 minutes or class 1 for 30 minutes or more.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Most frequent baseline	0.9122	0.0000	0.0000	0.0000	0.5000
Balanced logistic regression	0.9176	0.5183	0.8647	0.6482	0.9622
Optimized gradient boosting	0.9580	0.7954	0.7029	0.7463	0.9745

The optimized model uses a tuned decision threshold of 0.80. It improves precision to 0.7954 and accuracy to 0.9580 while keeping recall at 0.7029 on the held-out test set.

8. How to Predict a New Taxi Trip

Open notebooks/predict_new_taxi_trip.ipynb in Colab. Change the input values in the New Taxi Trip Input cell and run it. The result shows predicted_long_trip_class and long_trip_probability.

```
# Meaning of the prediction output
predicted_long_trip_class = 0 # Under 30 minutes
predicted_long_trip_class = 1 # 30 minutes or more

# Example input fields
pickup_datetime = "2024-01-15 17:30:00"
trip_distance = 14.2
passenger_count = 2
vendor_id = 2
ratecode_id = 1
pu_location_id = 132
do_location_id = 48
```

9. Generated Outputs

- reports/metrics.json - dataset, target distribution, split summary, and model metrics.
- reports/preprocessing_audit.csv - row counts before and after each cleaning step.
- reports/model_metrics.csv - compact baseline, initial logistic, and optimized model table.
- reports/tuning_results.csv - validation hyperparameter and threshold tuning results.
- reports/model_feature_importance.csv - permutation importance for the optimized model.
- reports/model_coefficients.csv - reference coefficients from the initial logistic model.
- reports/figures/ - charts and confusion matrix.
- reports/NYC_Taxi_AI_Analytics_APA7_Report.docx - APA 7 written report.

10. Key Figures

Figure 1. Distribution of cleaned trip duration.

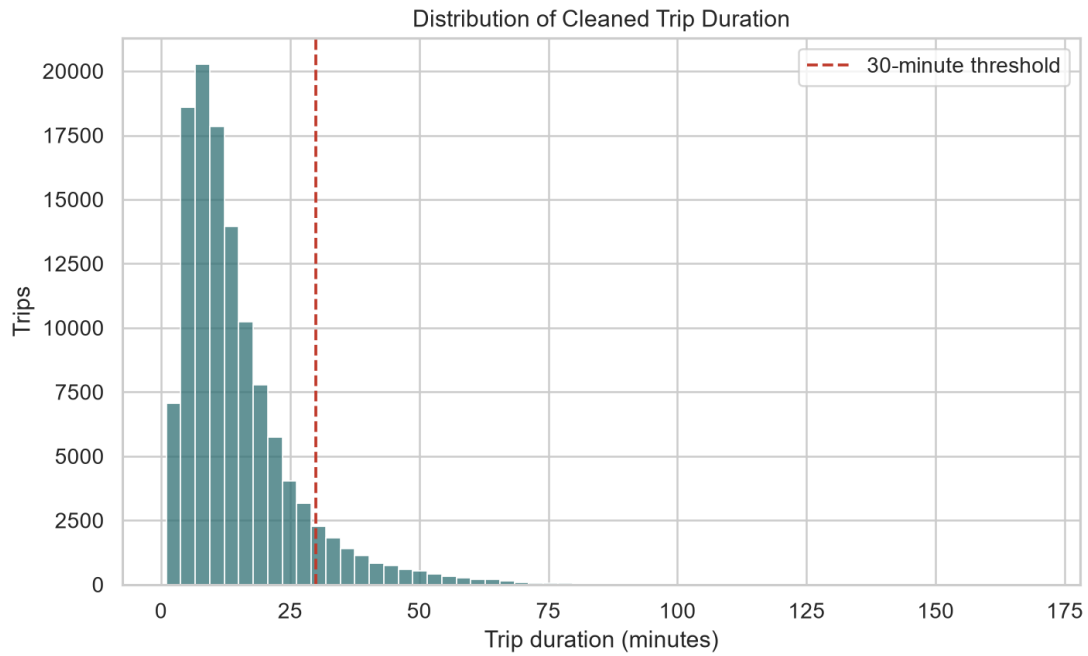


Figure 2. Long-trip rate by pickup hour.

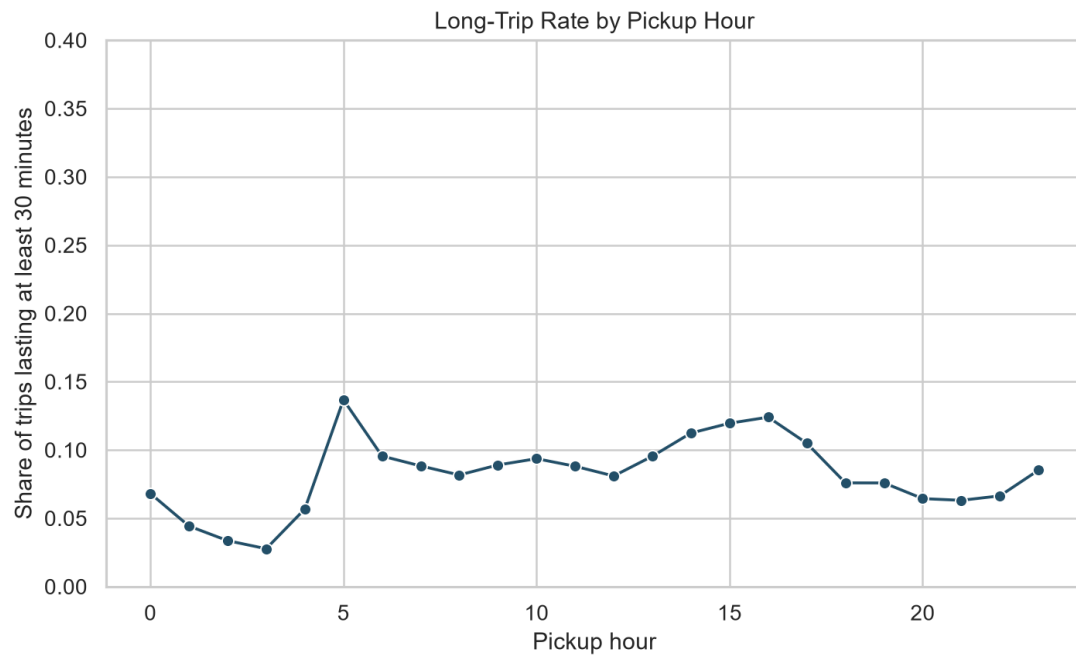


Figure 3. Optimized model confusion matrix.

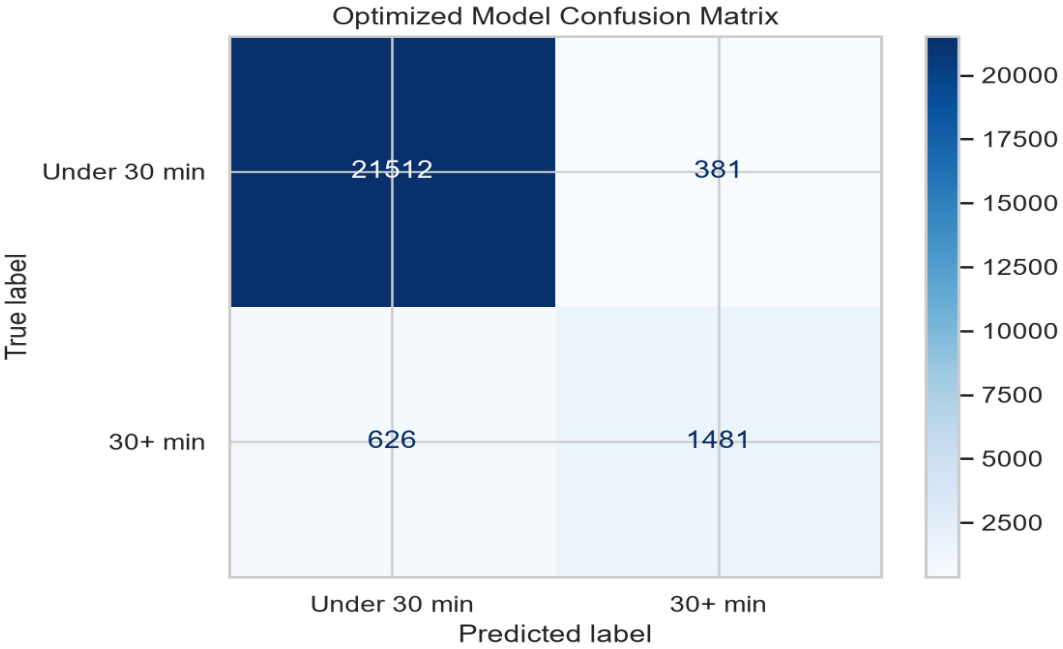
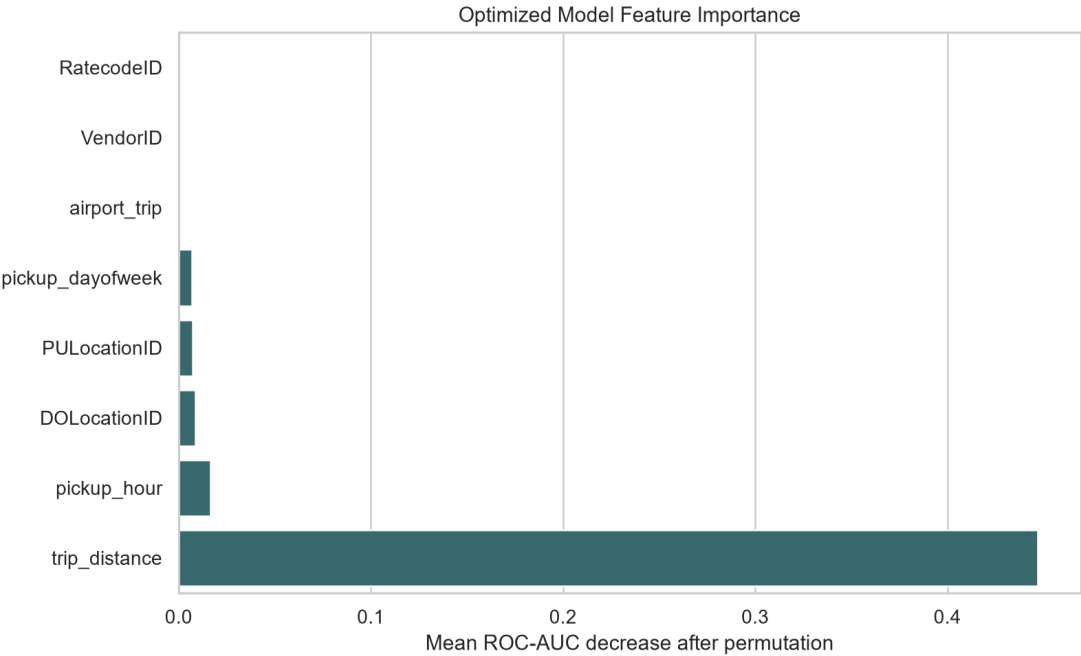


Figure 4. Optimized model feature importance.



11. Core Prediction Notebook Code

The prediction notebook contains setup, dependency verification, model creation/loading, feature construction, and new-trip prediction cells. The most important code cells are included below.

Prediction notebook code cell 1

```
from pathlib import Path
import os
import shutil
import subprocess
import sys

repo_url = "https://github.com/soumithkumara/project_day_1.git"
repo_branch = "project_day_2"
repo_dir = Path("/content/project_day_1")

# If the notebook was opened directly from GitHub, clone the full repo first.
if not Path("requirements.txt").exists():
    if repo_dir.exists():
        shutil.rmtree(repo_dir)
        subprocess.run(["git", "clone", "--branch", repo_branch, repo_url, str(repo_dir)],
                        check=True)
        os.chdir(repo_dir)
    elif Path(".git").exists():
        subprocess.run(["git", "fetch", "origin", repo_branch], check=True)
        subprocess.run(["git", "checkout", repo_branch], check=True)
        subprocess.run(["git", "pull", "origin", repo_branch], check=True)

print(f"Working directory: {Path.cwd()}")
print(f"Active branch: {subprocess.check_output(['git', 'branch', '--show-current'],
        text=True).strip()}")
```

Prediction notebook code cell 2

```
import importlib

required_packages = {
    "pandas": "pandas",
    "numpy": "numpy",
    "sklearn": "scikit-learn",
    "joblib": "joblib",
    "pyarrow": "pyarrow",
    "matplotlib": "matplotlib",
    "seaborn": "seaborn",
}

missing_packages = []
for module_name, package_name in required_packages.items():
    try:
        importlib.import_module(module_name)
    except ModuleNotFoundError:
        missing_packages.append(package_name)
    except Exception as exc:
        raise RuntimeError(
            "The Colab runtime has a partially upgraded or inconsistent Python package stack."
            "Choose Runtime > Restart session, then run this notebook from the top."
        ) from exc

if missing_packages:
    print("Installing missing packages:", missing_packages)
    subprocess.run([sys.executable, "-m", "pip", "install", *missing_packages], check=True)
    print("If Colab asks you to restart the runtime, restart and then run all cells again.")
else:
    print("All required packages are already available. No package reinstall needed.")
```

Prediction notebook code cell 3

```
model_path = Path("models/long_trip_optimized_model.joblib")
metrics_path = Path("reports/metrics.json")

if not model_path.exists() or not metrics_path.exists():
    print("Optimized model or metrics not found. Running the training pipeline first...")
    subprocess.run([sys.executable, "src/nyc_taxi_long_trip_model.py"], check=True)
else:
    print(f"Using existing optimized model: {model_path}")

print(f"Model ready: {model_path.exists()}")
print(f"Metrics ready: {metrics_path.exists()}")
```

Prediction notebook code cell 4

```
import json

import joblib
import pandas as pd

model = joblib.load(model_path)
metrics = json.loads(metrics_path.read_text())
DECISION_THRESHOLD = metrics["model_optimization"]["decision_threshold"]
print(f"Using tuned long-trip threshold: {DECISION_THRESHOLD:.2f}")

# TLC location IDs for Newark, JFK, and LaGuardia airport zones.
AIRPORT_ZONE_IDS = {1, 132, 138}

def build_trip_features(
    pickup_datetime,
    trip_distance,
    passenger_count,
    vendor_id,
    ratecode_id,
    pu_location_id,
    do_location_id,
):
    pickup_datetime = pd.to_datetime(pickup_datetime)

    return pd.DataFrame([
        "trip_distance": float(trip_distance),
        "passenger_count": int(passenger_count),
        "pickup_hour": pickup_datetime.hour,
        "pickup_dayofweek": pickup_datetime.dayofweek,
        "is_weekend": int(pickup_datetime.dayofweek in [5, 6]),
        "airport_trip": int(
            int(pu_location_id) in AIRPORT_ZONE_IDS
            or int(do_location_id) in AIRPORT_ZONE_IDS
        ),
        "VendorID": str(vendor_id),
        "RatecodeID": str(ratecode_id),
        "PULocationID": str(pu_location_id),
        "DOLocationID": str(do_location_id),
    ])

def predict_long_trip(
    pickup_datetime,
    trip_distance,
    passenger_count,
    vendor_id,
    ratecode_id,
    pu_location_id,
    do_location_id,
):
    features = build_trip_features(
        pickup_datetime=pickup_datetime,
        trip_distance=trip_distance,
        passenger_count=passenger_count,
        vendor_id=vendor_id,
        ratecode_id=ratecode_id,
        pu_location_id=pu_location_id,
        do_location_id=do_location_id,
    )

    probability = float(model.predict_proba(features)[0][1])
    prediction = int(probability >= DECISION_THRESHOLD)

    return features, pd.DataFrame([
        "prediction_label": "Long trip: 30 minutes or more" if prediction == 1 else "Short
        trip: under 30 minutes",
        "predicted_long_trip_class": prediction,
        "long_trip_probability": round(probability, 4),
        "long_trip_probability_percent": f"{probability:.2%}",
        "decision_threshold": DECISION_THRESHOLD,
    ]])
```

Prediction notebook code cell 5

```
# @title New Taxi Trip Input
pickup_datetime = "2024-01-15 17:30:00" # @param {type:"string"}
trip_distance = 14.2 # @param {type:"number"}
passenger_count = 2 # @param {type:"integer"}
vendor_id = 2 # @param {type:"integer"}
ratecode_id = 1 # @param {type:"integer"}
pu_location_id = 132 # @param {type:"integer"}
do_location_id = 48 # @param {type:"integer"}
```

```

features, prediction_result = predict_long_trip(
    pickup_datetime=pickup_datetime,
    trip_distance=trip_distance,
    passenger_count=passenger_count,
    vendor_id=vendor_id,
    ratecode_id=ratecode_id,
    pu_location_id=pu_location_id,
    do_location_id=do_location_id,
)

print("Model input features:")
display(features)

print("Prediction:")
display(prediction_result)

```

Prediction notebook code cell 6

```

new_trips = [
    {
        "pickup_datetime": "2024-01-15 17:30:00",
        "trip_distance": 14.2,
        "passenger_count": 2,
        "vendor_id": 2,
        "ratecode_id": 1,
        "pu_location_id": 132,
        "do_location_id": 48,
    },
    {
        "pickup_datetime": "2024-01-16 10:15:00",
        "trip_distance": 1.4,
        "passenger_count": 1,
        "vendor_id": 2,
        "ratecode_id": 1,
        "pu_location_id": 237,
        "do_location_id": 236,
    },
]

batch_results = []
for i, trip in enumerate(new_trips, start=1):
    _, result = predict_long_trip(**trip)
    row = {"trip_number": i, **trip, **result.iloc[0].to_dict()}
    batch_results.append(row)

display(pd.DataFrame(batch_results))

```

12. Complete Main Pipeline Source Code

The full training, preprocessing, evaluation, chart-generation, and artifact-writing pipeline is included below for submission review.

```
"""Build an initial NYC taxi long-trip risk model.

The pipeline downloads the official NYC TLC yellow taxi parquet file, cleans
trip records, creates features, trains an interpretable classifier, and writes
figures/metrics for the assignment report.
"""

from __future__ import annotations

import argparse
import json
import math
import urllib.request
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    fbeta_score,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler

PROJECT_ROOT = Path(__file__).resolve().parents[1]
DEFAULT_DATA_URL = (
    "https://d37ci6vzurychx.cloudfront.net/trip-data/"
    "yellow_tripdata_2024-01.parquet"
)
DEFAULT_RAW_PATH = PROJECT_ROOT / "data" / "raw" / "yellow_tripdata_2024-01.parquet"
DEFAULT_REPORT_DIR = PROJECT_ROOT / "reports"
DEFAULT_MODEL_DIR = PROJECT_ROOT / "models"
RANDOM_STATE = 42

RAW_COLUMNS = [
    "VendorID",
    "tpep_pickup_datetime",
    "tpep_dropoff_datetime",
    "passenger_count",
    "trip_distance",
    "RatecodeID",
    "PULocationID",
    "DOLocationID",
    "payment_type",
    "fare_amount",
    "extra",
    "mta_tax",
    "tip_amount",
    "tolls_amount",
    "improvement_surcharge",
    "total_amount",
    "congestion_surcharge",
    "Airport_fee",
]

NUMERIC_FEATURES = [
    "trip_distance",
    "passenger_count",
```

```

    "pickup_hour",
    "pickup_dayofweek",
    "is_weekend",
    "airport_trip",
]

CATEGORICAL_FEATURES = [
    "VendorID",
    "RatecodeID",
    "PULocationID",
    "DOLocationID",
]

FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES
TARGET = "long_trip"
INITIAL_MODEL_NAME = "Balanced logistic regression"
FINAL_MODEL_NAME = "Optimized histogram gradient boosting"
DECISION_RECALL_FLOOR = 0.72
DECISION_BETA = 0.5

HGB_TUNING_CANDIDATES: list[dict[str, Any]] = [
    {
        "candidate": "hgb_depth31_weight4",
        "max_iter": 250,
        "learning_rate": 0.06,
        "max_leaf_nodes": 31,
        "l2_regularization": 0.0,
        "class_weight": {0: 1, 1: 4},
    },
    {
        "candidate": "hgb_depth63_weight4",
        "max_iter": 300,
        "learning_rate": 0.05,
        "max_leaf_nodes": 63,
        "l2_regularization": 0.01,
        "class_weight": {0: 1, 1: 4},
    },
    {
        "candidate": "hgb_depth31_balanced",
        "max_iter": 300,
        "learning_rate": 0.05,
        "max_leaf_nodes": 31,
        "l2_regularization": 0.1,
        "class_weight": "balanced",
    },
    {
        "candidate": "hgb_depth63_weight6",
        "max_iter": 200,
        "learning_rate": 0.08,
        "max_leaf_nodes": 63,
        "l2_regularization": 0.1,
        "class_weight": {0: 1, 1: 6},
    },
    {
        "candidate": "hgb_depth31_weight3",
        "max_iter": 400,
        "learning_rate": 0.03,
        "max_leaf_nodes": 31,
        "l2_regularization": 0.01,
        "class_weight": {0: 1, 1: 3},
    },
]

# TLC location IDs for Newark, JFK, and LaGuardia airport zones.
AIRPORT_ZONE_IDS = {1, 132, 138}

@dataclass(frozen=True)
class PipelineConfig:
    data_url: str
    raw_path: Path
    report_dir: Path
    model_dir: Path
    sample_size: int
    force_download: bool

def download_file(url: str, output_path: Path, force: bool = False) -> Path:
    """Download a file unless it already exists locally."""
    output_path.parent.mkdir(parents=True, exist_ok=True)
    if output_path.exists() and not force:
        return output_path

    def report_hook(block_num: int, block_size: int, total_size: int) -> None:
        if total_size <= 0:

```

```

        return
    downloaded = block_num * block_size
    percent = min(100, downloaded * 100 / total_size)
    if block_num % 100 == 0 or math.isclose(percent, 100):
        print(f"Downloading data: {percent:5.1f}%")

    urllib.request.urlretrieve(url, output_path, reporthook=report_hook)
    return output_path

def load_raw_data(path: Path) -> pd.DataFrame:
    """Read only the columns needed for this assignment."""
    return pd.read_parquet(path, columns=RAW_COLUMNS)

def _audit_filter(
    df: pd.DataFrame,
    mask: pd.Series,
    step: str,
    description: str,
    audit_rows: list[dict[str, Any]],
    raw_rows: int,
) -> pd.DataFrame:
    """Apply one preprocessing filter and record its row impact."""
    rows_before = len(df)
    filtered = df.loc[mask].copy()
    rows_after = len(filtered)
    audit_rows.append(
        {
            "step": step,
            "description": description,
            "rows_before": rows_before,
            "rows_after": rows_after,
            "rows_removed": rows_before - rows_after,
            "step_removed_pct": round(
                (rows_before - rows_after) / rows_before * 100, 4
            ),
            "retained_pct_of_raw": round((rows_after / raw_rows) * 100, 4),
        }
    )
    return filtered

def preprocess_with_audit(
    raw: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """Clean TLC records, create modeling features, and document each change."""
    df = raw.copy()
    raw_rows = len(df)
    audit_rows: list[dict[str, Any]] = [
        {
            "step": "01_raw_load",
            "description": "Loaded selected NYC TLC yellow taxi columns from parquet.",
            "rows_before": raw_rows,
            "rows_after": raw_rows,
            "rows_removed": 0,
            "step_removed_pct": 0.0,
            "retained_pct_of_raw": 100.0,
        }
    ]

    df["tpep_pickup_datetime"] = pd.to_datetime(df["tpep_pickup_datetime"])
    df["tpep_dropoff_datetime"] = pd.to_datetime(df["tpep_dropoff_datetime"])
    df["trip_duration_min"] = (
        df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"]
    ).dt.total_seconds() / 60
    audit_rows.append(
        {
            "step": "02_derive_duration",
            "description": "Converted pickup/dropoff timestamps and created trip_duration_min.",
            "rows_before": len(df),
            "rows_after": len(df),
            "rows_removed": 0,
            "step_removed_pct": 0.0,
            "retained_pct_of_raw": round((len(df) / raw_rows) * 100, 4),
        }
    )

    df = _audit_filter(
        df,
        (df["tpep_pickup_datetime"] >= "2024-01-01")
    )

```

```

        & (df["tpep_pickup_datetime"] < "2024-02-01"),
        "03_filter_pickup_month",
        "Kept trips with pickup timestamps in January 2024.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["trip_duration_min"].between(1, 180),
        "04_filter_duration",
        "Removed trips shorter than 1 minute or longer than 180 minutes.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["trip_distance"].between(0.1, 60),
        "05_filter_distance",
        "Removed trips with zero, near-zero, negative, or extreme distances.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["passenger_count"].between(1, 6),
        "06_filter_passengers",
        "Kept trips with passenger_count from 1 to 6.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["fare_amount"] > 0,
        "07_filter_positive_fare",
        "Removed trips with non-positive fare_amount.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["total_amount"] > 0,
        "08_filter_positive_total",
        "Removed trips with non-positive total_amount.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["RatecodeID"].between(1, 6),
        "09_filter_standard_ratecode",
        "Kept standard TLC rate codes 1 through 6.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["PULocationID"].between(1, 265),
        "10_filter_pickup_zone",
        "Kept valid TLC pickup location IDs from 1 to 265.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["DOLocationID"].between(1, 265),
        "11_filter_dropoff_zone",
        "Kept valid TLC dropoff location IDs from 1 to 265.",
        audit_rows,
        raw_rows,
    )

    df["pickup_hour"] = df["tpep_pickup_datetime"].dt.hour
    df["pickup_dayofweek"] = df["tpep_pickup_datetime"].dt.dayofweek
    df["is_weekend"] = df["pickup_dayofweek"].isin([5, 6]).astype(int)
    df["airport_trip"] = (
        df["PULocationID"].isin(AIRPORT_ZONE_IDS)
        | df["DOLocationID"].isin(AIRPORT_ZONE_IDS)
    ).astype(int)
    df[TARGET] = (df["trip_duration_min"] >= 30).astype(int)

    for column in CATEGORICAL_FEATURES:
        df[column] = df[column].astype("Int64").astype(str)

    audit_rows.append(
        {

```

```

        "step": "12_feature_engineering",
        "description": "Created time, weekend, airport, target, and categorical model
            fields.",
        "rows_before": len(df),
        "rows_after": len(df),
        "rows_removed": 0,
        "step_removed_pct": 0.0,
        "retained_pct_of_raw": round((len(df) / raw_rows) * 100, 4),
    }
)

feature_rows = [
    {
        "column": "trip_duration_min",
        "type": "derived numeric",
        "source": "tpep_dropoff_datetime - tpep_pickup_datetime",
        "purpose": "Used for cleaning and to create the long_trip target; not used as a
            model input.",
    },
    {
        "column": "pickup_hour",
        "type": "derived numeric",
        "source": "hour from tpep_pickup_datetime",
        "purpose": "Captures daily travel and congestion cycles.",
    },
    {
        "column": "pickup_dayofweek",
        "type": "derived numeric",
        "source": "day of week from tpep_pickup_datetime, Monday=0",
        "purpose": "Captures weekday/weekend and commute-pattern differences.",
    },
    {
        "column": "is_weekend",
        "type": "derived binary",
        "source": "pickup_dayofweek in Saturday or Sunday",
        "purpose": "Flags weekend operating patterns.",
    },
    {
        "column": "airport_trip",
        "type": "derived binary",
        "source": "pickup or dropoff zone in Newark, JFK, or LaGuardia TLC IDs",
        "purpose": "Flags trips involving major airport zones.",
    },
    {
        "column": "TARGET",
        "type": "target binary",
        "source": "trip_duration_min >= 30",
        "purpose": "Prediction target: 1 for long trips, 0 otherwise.",
    },
    {
        "column": " ".join(CATEGORICAL_FEATURES),
        "type": "converted categorical",
        "source": "integer IDs converted to strings",
        "purpose": "Prevents ID values from being treated as ordered numeric quantities.",
    },
]

return df, pd.DataFrame(audit_rows), pd.DataFrame(feature_rows)

def clean_and_engineer_features(raw: pd.DataFrame) -> pd.DataFrame:
    """Clean TLC records and create modeling features."""
    df, _, _ = preprocess_with_audit(raw)
    return df

def make_logistic_model() -> Pipeline:
    """Create the initial interpretable logistic classification pipeline."""
    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            (
                "onehot",
                OneHotEncoder(handle_unknown="ignore", min_frequency=50),
            ),
        ]
    )
    preprocessor = ColumnTransformer(
        transformers=[

```

```

        ("num", numeric_transformer, NUMERIC_FEATURES),
        ("cat", categorical_transformer, CATEGORICAL_FEATURES),
    ]
)
classifier = LogisticRegression(
    class_weight="balanced",
    max_iter=1000,
    random_state=RANDOM_STATE,
)
return Pipeline(steps=[("preprocess", preprocessor), ("model", classifier)])

def make_hist_gradient_boosting_model(params: dict[str, Any]) -> Pipeline:
    """Create the optimized model family used during hyperparameter tuning."""
    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            (
                "ordinal",
                OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1),
            ),
        ]
    )
    preprocessor = ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, NUMERIC_FEATURES),
            ("cat", categorical_transformer, CATEGORICAL_FEATURES),
        ],
        verbose_feature_names_out=False,
    )
    categorical_indices = list(range(len(NUMERIC_FEATURES), len(FEATURES)))
    model_params = {key: value for key, value in params.items() if key != "candidate"}
    classifier = HistGradientBoostingClassifier(
        categorical_features=categorical_indices,
        random_state=RANDOM_STATE,
        **model_params,
    )
    return Pipeline(steps=[("preprocess", preprocessor), ("model", classifier)])

def make_model() -> Pipeline:
    """Create the default optimized classification pipeline."""
    return make_hist_gradient_boosting_model(HGB_TUNING_CANDIDATES[1])

def evaluate_predictions(
    y_true: pd.Series,
    y_pred: np.ndarray,
    y_score: np.ndarray,
    threshold: float | None = None,
) -> dict[str, Any]:
    """Calculate core classification metrics."""
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    metrics = {
        "accuracy": round(float(accuracy_score(y_true, y_pred)), 4),
        "precision": round(float(precision_score(y_true, y_pred, zero_division=0)), 4),
        "recall": round(float(recall_score(y_true, y_pred, zero_division=0)), 4),
        "f1": round(float(f1_score(y_true, y_pred, zero_division=0)), 4),
        "f0.5": round(
            float(fbeta_score(y_true, y_pred, beta=0.5, zero_division=0)),
            4,
        ),
        "roc_auc": round(float(roc_auc_score(y_true, y_score)), 4),
        "true_negative": int(tn),
        "false_positive": int(fp),
        "false_negative": int(fn),
        "true_positive": int(tp),
    }
    if threshold is not None:
        metrics["decision_threshold"] = round(float(threshold), 4)
    return metrics

def threshold_search(
    y_true: pd.Series,
    y_score: np.ndarray,
    recall_floor: float = DECISION_RECALL_FLOOR,
    beta: float = DECISION_BETA,
) -> tuple[float, pd.DataFrame]:
    """Choose a probability threshold on validation data."""

```

```

rows: list[dict[str, Any]] = []
for threshold in np.round(np.arange(0.05, 0.951, 0.01), 2):
    y_pred = (y_score >= threshold).astype(int)
    precision = precision_score(y_true, y_pred, zero_division=0)
    recall = recall_score(y_true, y_pred, zero_division=0)
    rows.append(
        {
            "threshold": float(threshold),
            "accuracy": float(accuracy_score(y_true, y_pred)),
            "precision": float(precision),
            "recall": float(recall),
            "f1": float(f1_score(y_true, y_pred, zero_division=0)),
            "f0_5": float(
                fbeta_score(y_true, y_pred, beta=beta, zero_division=0)
            ),
            "predicted_positive_rate": float(np.mean(y_pred)),
        }
    )

threshold_frame = pd.DataFrame(rows)
feasible = threshold_frame[threshold_frame["recall"] >= recall_floor]
selection_frame = feasible if not feasible.empty else threshold_frame
selected = selection_frame.sort_values(
    ["f0_5", "accuracy", "precision"],
    ascending=[False, False, False],
).iloc[0]
return float(selected["threshold"]), threshold_frame

def tune_hist_gradient_boosting(
    x_train: pd.DataFrame,
    y_train: pd.Series,
    x_validation: pd.DataFrame,
    y_validation: pd.Series,
) -> tuple[Pipeline, float, dict[str, Any], pd.DataFrame]:
    """Tune gradient boosting hyperparameters and decision threshold."""
    tuning_rows: list[dict[str, Any]] = []
    best_model: Pipeline | None = None
    best_threshold = 0.5
    best_candidate: dict[str, Any] | None = None
    best_score_key = (-np.inf, -np.inf, -np.inf)

    for candidate in HGB_TUNING_CANDIDATES:
        model = make_hist_gradient_boosting_model(candidate)
        model.fit(x_train, y_train)
        validation_score = model.predict_proba(x_validation)[0, 1]
        threshold, _ = threshold_search(y_validation, validation_score)
        validation_pred = (validation_score >= threshold).astype(int)
        metrics = evaluate_predictions(
            y_validation,
            validation_pred,
            validation_score,
            threshold=threshold,
        )
        row = {
            "candidate": candidate["candidate"],
            "model": FINAL_MODEL_NAME,
            "selected_threshold": round(float(threshold), 4),
            "max_iter": candidate["max_iter"],
            "learning_rate": candidate["learning_rate"],
            "max_leaf_nodes": candidate["max_leaf_nodes"],
            "l2_regularization": candidate["l2_regularization"],
            "class_weight": json.dumps(candidate["class_weight"]),
            "selection_metric": "validation_f0_5_with_recall_floor",
            "validation_recall_floor": DECISION_RECALL_FLOOR,
            **{f"validation_{key}": value for key, value in metrics.items()},
        }
        tuning_rows.append(row)
        score_key = (
            metrics["f0_5"],
            metrics["accuracy"],
            metrics["precision"],
        )
        if score_key > best_score_key:
            best_score_key = score_key
            best_model = model
            best_threshold = threshold
            best_candidate = candidate

    if best_model is None or best_candidate is None:
        raise RuntimeError("No tuned model candidate was selected.")

    tuning_frame = pd.DataFrame(tuning_rows)
    tuning_frame["selected"] = tuning_frame["candidate"] == best_candidate["candidate"]
    return best_model, best_threshold, best_candidate, tuning_frame

```

```

def coefficient_summary(model: Pipeline) -> pd.DataFrame:
    """Return sorted feature coefficients from the fitted logistic model."""
    preprocessor = model.named_steps["preprocess"]
    classifier = model.named_steps["model"]
    feature_names = preprocessor.get_feature_names_out()
    coefficients = classifier.coef_.ravel()
    frame = pd.DataFrame(
        {
            "feature": feature_names,
            "coefficient": coefficients,
            "absolute_coefficient": np.abs(coefficients),
        }
    )
    frame["feature"] = (
        frame["feature"]
        .str.replace("num_", "", regex=False)
        .str.replace("cat_", "", regex=False)
        .str.replace("onehot_", "", regex=False)
    )
    frame["direction"] = np.where(
        frame["coefficient"] >= 0,
        "Higher long-trip risk",
        "Lower long-trip risk",
    )
    return frame.sort_values("absolute_coefficient", ascending=False)

def feature_importance_summary(
    model: Pipeline,
    x_test: pd.DataFrame,
    y_test: pd.Series,
    max_rows: int = 15_000,
) -> pd.DataFrame:
    """Estimate final model feature importance with held-out permutation tests."""
    if len(x_test) > max_rows:
        x_importance = x_test.sample(n=max_rows, random_state=RANDOM_STATE)
        y_importance = y_test.loc[x_importance.index]
    else:
        x_importance = x_test
        y_importance = y_test

    result = permutation_importance(
        model,
        x_importance,
        y_importance,
        scoring="roc_auc",
        n_repeats=5,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )
    frame = pd.DataFrame(
        {
            "feature": list(x_importance.columns),
            "importance_mean": result.importances_mean,
            "importance_std": result.importances_std,
            "scoring": "roc_auc",
        }
    )
    return frame.sort_values("importance_mean", ascending=False)

def create_figures(
    df: pd.DataFrame,
    model: Pipeline,
    y_test: pd.Series,
    y_pred: np.ndarray,
    feature_importance: pd.DataFrame,
    report_dir: Path,
    initial_model: Pipeline | None = None,
) -> None:
    """Create EDA and model evaluation figures."""
    figure_dir = report_dir / "figures"
    figure_dir.mkdir(parents=True, exist_ok=True)
    sns.set_theme(style="whitegrid")

    plt.figure(figsize=(8, 5))
    sns.histplot(df["trip_duration_min"], bins=60, color="#2f6f73")
    plt.axvline(30, color="#c03a2b", linestyle="--", label="30-minute threshold")
    plt.title("Distribution of Cleaned Trip Duration")
    plt.xlabel("Trip duration (minutes)")
    plt.ylabel("Trips")
    plt.legend()
    plt.tight_layout()
    plt.savefig(figure_dir / "duration_distribution.png", dpi=180)
    plt.close()

```

```

plot_sample = df.sample(n=min(len(df), 20000), random_state=RANDOM_STATE)
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=plot_sample,
    x="trip_distance",
    y="trip_duration_min",
    hue=TARGET,
    alpha=0.35,
    palette={0: "#5677a6", 1: "#d1683c"},
    linewidth=0,
)
plt.title("Trip Distance vs. Duration")
plt.xlabel("Trip distance (miles)")
plt.ylabel("Trip duration (minutes)")
plt.tight_layout()
plt.savefig(figure_dir / "distance_vs_duration.png", dpi=180)
plt.close()

hourly = (
    df.groupby("pickup_hour", as_index=False)[TARGET]
    .mean()
    .rename(columns={TARGET: "long_trip_rate"})
)
plt.figure(figsize=(8, 5))
sns.lineplot(
    data=hourly,
    x="pickup_hour",
    y="long_trip_rate",
    marker="o",
    color="#234f68",
)
plt.title("Long-Trip Rate by Pickup Hour")
plt.xlabel("Pickup hour")
plt.ylabel("Share of trips lasting at least 30 minutes")
plt.ylim(0, max(0.4, hourly["long_trip_rate"].max() + 0.03))
plt.tight_layout()
plt.savefig(figure_dir / "hourly_long_trip_rate.png", dpi=180)
plt.close()

cm = confusion_matrix(y_test, y_pred)
display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Under 30 min", "30+ min"],
)
display.plot(cmap="Blues", values_format="d")
plt.title("Optimized Model Confusion Matrix")
plt.tight_layout()
plt.savefig(figure_dir / "confusion_matrix.png", dpi=180)
plt.close()

positive_importance = feature_importance[feature_importance["importance_mean"] > 0]
importance_frame = positive_importance.head(10).sort_values(
    "importance_mean",
    ascending=True,
)
plt.figure(figsize=(9, 5.5))
sns.barplot(
    data=importance_frame,
    y="feature",
    x="importance_mean",
    color="#2f6f73",
)
plt.title("Optimized Model Feature Importance")
plt.xlabel("Mean ROC-AUC decrease after permutation")
plt.ylabel("")
plt.tight_layout()
plt.savefig(figure_dir / "feature_importance.png", dpi=180)
plt.close()

if initial_model is None:
    return

coefficient_frame = coefficient_summary(initial_model).head(15)
plt.figure(figsize=(9, 6))
sns.barplot(
    data=coefficient_frame,
    y="feature",
    x="coefficient",
    hue="direction",
    dodge=False,
    palette={
        "Higher long-trip risk": "#d1683c",
        "Lower long-trip risk": "#5677a6",
    },
)

```

```

plt.axvline(0, color="black", linewidth=0.8)
plt.title("Initial Logistic Regression Coefficients")
plt.xlabel("Standardized coefficient")
plt.ylabel("")
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig(figure_dir / "top_coefficients.png", dpi=180)
plt.close()

def write_outputs(
    df: pd.DataFrame,
    raw_rows: int,
    clean_rows_before_sampling: int,
    preprocessing_audit: pd.DataFrame,
    feature_summary: pd.DataFrame,
    split_summary: dict[str, Any],
    baseline_metrics: dict[str, Any],
    initial_metrics: dict[str, Any],
    final_metrics: dict[str, Any],
    selected_candidate: dict[str, Any],
    final_threshold: float,
    tuning_results: pd.DataFrame,
    report: str,
    final_model: Pipeline,
    initial_model: Pipeline,
    feature_importance: pd.DataFrame,
    report_dir: Path,
    model_dir: Path,
) -> None:
    """Persist metrics, feature notes, and the trained model."""
    report_dir.mkdir(parents=True, exist_ok=True)
    model_dir.mkdir(parents=True, exist_ok=True)

    summary = {
        "dataset": {
            "source": DEFAULT_DATA_URL,
            "raw_rows": int(raw_rows),
            "clean_rows_before_sampling": int(clean_rows_before_sampling),
            "model_sample_rows": int(len(df)),
            "long_trip_rate": round(float(df[TARGET].mean()), 4),
            "median_duration_min": round(float(df["trip_duration_min"].median()), 2),
            "median_distance_miles": round(float(df["trip_distance"].median()), 2),
        },
        "target_distribution": {
            "under_30_min": int((df[TARGET] == 0).sum()),
            "long_30_plus_min": int((df[TARGET] == 1).sum()),
            "long_trip_rate": round(float(df[TARGET].mean()), 4),
        },
        "model_inputs": {
            "numeric_features": NUMERIC_FEATURES,
            "categorical_features": CATEGORICAL_FEATURES,
            "target": TARGET,
            "positive_class": "long_trip = 1 means trip duration is at least 30 minutes",
        },
        "train_test_split": split_summary,
        "model_optimization": {
            "selected_model": FINAL_MODEL_NAME,
            "selected_candidate": selected_candidate,
            "decision_threshold": round(float(final_threshold), 4),
            "selection_metric": "validation F0.5 with recall >= 0.72",
            "reason": (
                "F0.5 emphasizes precision while the recall floor keeps the model "
                "useful for long-trip planning."
            ),
        },
        "baseline_most_frequent": baseline_metrics,
        "initial_logistic_regression": initial_metrics,
        "logistic_regression": initial_metrics,
        "optimized_hist_gradient_boosting": final_metrics,
        "optimized_model": final_metrics,
        "classification_report": report,
    }

    with (report_dir / "metrics.json").open("w", encoding="utf-8") as file:
        json.dump(summary, file, indent=2)

    preprocessing_audit.to_csv(report_dir / "preprocessing_audit.csv", index=False)
    feature_summary.to_csv(report_dir / "feature_engineering_summary.csv", index=False)
    tuning_results.to_csv(report_dir / "tuning_results.csv", index=False)

    pd.DataFrame(
        [
            {"model": "Most frequent baseline", **baseline_metrics},
            {"model": INITIAL_MODEL_NAME, **initial_metrics},
        ]
    )

```

```

        {"model": FINAL_MODEL_NAME, **final_metrics},
    ]
).to_csv(report_dir / "model_metrics.csv", index=False)

pd.DataFrame(
    [
        {
            "target_value": 0,
            "label": "Under 30 minutes",
            "rows": int((df[TARGET] == 0).sum()),
            "share": round(float((df[TARGET] == 0).mean()), 4),
        },
        {
            "target_value": 1,
            "label": "30+ minutes",
            "rows": int((df[TARGET] == 1).sum()),
            "share": round(float((df[TARGET] == 1).mean()), 4),
        },
    ]
).to_csv(report_dir / "target_distribution.csv", index=False)

coefficient_summary(initial_model).to_csv(
    report_dir / "model_coefficients.csv",
    index=False,
)
feature_importance.to_csv(report_dir / "model_feature_importance.csv", index=False)
joblib.dump(final_model, model_dir / "long_trip_optimized_model.joblib")

def run_pipeline(config: PipelineConfig) -> dict[str, Any]:
    raw_path = download_file(config.data_url, config.raw_path, config.force_download)
    raw = load_raw_data(raw_path)
    raw_rows = len(raw)
    df, preprocessing_audit, feature_summary = preprocess_with_audit(raw)
    clean_rows = len(df)
    if config.sample_size and len(df) > config.sample_size:
        rows_before_sample = len(df)
        df = df.sample(n=config.sample_size, random_state=RANDOM_STATE)
        preprocessing_audit = pd.concat(
            [
                preprocessing_audit,
                pd.DataFrame(
                    [
                        {
                            "step": "13_model_sample",
                            "description": (
                                "Selected deterministic modeling sample with "
                                f"random_state={RANDOM_STATE}."
                            ),
                            "rows_before": rows_before_sample,
                            "rows_after": len(df),
                            "rows_removed": rows_before_sample - len(df),
                            "step_removed_pct": round(
                                ((rows_before_sample - len(df)) / rows_before_sample)
                                * 100,
                                4,
                            ),
                            "retained_pct_of_raw": round((len(df) / raw_rows) * 100, 4),
                        }
                    ]
                ),
            ],
            ignore_index=True,
        )

    x = df[FEATURES]
    y = df[TARGET]
    x_development, x_test, y_development, y_test = train_test_split(
        x,
        y,
        test_size=0.2,
        random_state=RANDOM_STATE,
        stratify=y,
    )
    x_train, x_validation, y_train, y_validation = train_test_split(
        x_development,
        y_development,
        test_size=0.25,
        random_state=RANDOM_STATE,
        stratify=y_development,
    )
    split_summary = {
        "split_method": "60/20/20 stratified train-validation-test split",
        "random_state": RANDOM_STATE,
        "train_rows": int(len(x_train)),
    }

```

```

    "validation_rows": int(len(x_validation)),
    "test_rows": int(len(x_test)),
    "train_long_trip_rate": round(float(y_train.mean()), 4),
    "validation_long_trip_rate": round(float(y_validation.mean()), 4),
    "test_long_trip_rate": round(float(y_test.mean()), 4),
}

baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(x_train, y_train)
baseline_pred = baseline.predict(x_test)
baseline_score = np.repeat(y_train.mean(), repeats=len(y_test))
baseline_metrics = evaluate_predictions(y_test, baseline_pred, baseline_score)

initial_model = make_logistic_model()
initial_model.fit(x_train, y_train)
initial_score = initial_model.predict_proba(x_test)[: , 1]
initial_pred = (initial_score >= 0.5).astype(int)
initial_metrics = evaluate_predictions(
    y_test,
    initial_pred,
    initial_score,
    threshold=0.5,
)

final_model, final_threshold, selected_candidate, tuning_results = (
    tune_hist_gradient_boosting(x_train, y_train, x_validation, y_validation)
)
y_score = final_model.predict_proba(x_test)[: , 1]
y_pred = (y_score >= final_threshold).astype(int)
final_metrics = evaluate_predictions(
    y_test,
    y_pred,
    y_score,
    threshold=final_threshold,
)

report = classification_report(
    y_test,
    y_pred,
    target_names=["Under 30 min", "30+ min"],
    digits=4,
)

feature_importance = feature_importance_summary(final_model, x_test, y_test)
create_figures(
    df,
    final_model,
    y_test,
    y_pred,
    feature_importance,
    config.report_dir,
    initial_model=initial_model,
)

write_outputs(
    df,
    raw_rows,
    clean_rows,
    preprocessing_audit,
    feature_summary,
    split_summary,
    baseline_metrics,
    initial_metrics,
    final_metrics,
    selected_candidate,
    final_threshold,
    tuning_results,
    report,
    final_model,
    initial_model,
    feature_importance,
    config.report_dir,
    config.model_dir,
)

return {
    "raw_rows": raw_rows,
    "clean_rows_before_sampling": clean_rows,
    "sample_rows": len(df),
    "long_trip_rate": round(float(df[TARGET].mean()), 4),
    "initial_metrics": initial_metrics,
    "optimized_metrics": final_metrics,
    "baseline_metrics": baseline_metrics,
    "selected_candidate": selected_candidate["candidate"],
    "decision_threshold": round(float(final_threshold), 4),
}

```

```
def parse_args() -> argparse.Namespace:
```

```

parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument("--data-url", default=DEFAULT_DATA_URL)
parser.add_argument("--raw-path", type=Path, default=DEFAULT_RAW_PATH)
parser.add_argument("--report-dir", type=Path, default=DEFAULT_REPORT_DIR)
parser.add_argument("--model-dir", type=Path, default=DEFAULT_MODEL_DIR)
parser.add_argument("--sample-size", type=int, default=120_000)
parser.add_argument("--force-download", action="store_true")
return parser.parse_args()

def main() -> None:
    args = parse_args()
    config = PipelineConfig(
        data_url=args.data_url,
        raw_path=args.raw_path,
        report_dir=args.report_dir,
        model_dir=args.model_dir,
        sample_size=args.sample_size,
        force_download=args.force_download,
    )
    result = run_pipeline(config)
    print(json.dumps(result, indent=2, default=str))

if __name__ == "__main__":
    main()

```

13. Troubleshooting Notes

- If Colab cannot find files, run `%cd /content/project_day_1` and then `!git pull origin project_day_2`.
- If `preprocessing_audit.csv` is missing, run `!python src/nyc_taxi_long_trip_model.py`.
- If the model file is missing, run the training pipeline; the model is generated locally as `models/long_trip_optimized_model.joblib`.
- If Colab shows a NumPy binary error, choose Runtime > Restart session and run the updated notebook from the top.
- Raw data and model binaries are intentionally not committed to GitHub because they are reproducible generated files.

14. References

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95.
<https://doi.org/10.1109/MCSE.2007.55>

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56-61. <https://doi.org/10.25080/Majora-92bf1922-00a>

New York City Taxi and Limousine Commission. (2024). TLC trip record data.
<https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://jmlr.org/papers/v12/pedregosa11a.html>

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.
<https://doi.org/10.21105/joss.03021>